

Behavioral Design Patterns in Java

Structure Code • Where to Apply • Practical Scenarios

Pattern	Core idea	Think
Command	Encapsulate a request as an object	What action?
Mediator	Centralize communication	Who coordinates?
Observer	Notify dependents of changes	Who needs to know?
State	Behavior depends on current state	What state am I in?
Strategy	Interchangeable algorithms	How should I do it?
Template Method	Fixed skeleton, customizable steps	Which steps vary?

1. Command Pattern

What it solves: Encapsulates a request/action as an object. It decouples the invoker (button, remote, queue) from the receiver that performs the work.

Structure / Java template

```
interface Command {
    void execute();
}

class ConcreteCommand implements Command {
    private Receiver receiver;

    ConcreteCommand(Receiver receiver) {
        this.receiver = receiver;
    }

    public void execute() {
        receiver.action();
    }
}

class Receiver {
    void action() {
        System.out.println("Performing action");
    }
}

class Invoker {
    private Command command;

    void setCommand(Command command) {
        this.command = command;
    }

    void pressButton() {
        command.execute();
    }
}
```

Where to apply: GUI buttons and menus • Remote controls • Undo/redo • Task queues • Logging/replaying actions

Typical scenarios

- Remote: TurnOnCommand → Television
- Editor: DeleteCommand → Document
- Queue: each job is a Command

2. Mediator Pattern

What it solves: Centralizes communication between collaborating objects so they do not need direct references to each other.

Structure / Java template

```
interface Mediator {
    void send(String message, Colleague sender);
}

abstract class Colleague {
    protected Mediator mediator;

    Colleague(Mediator mediator) {
        this.mediator = mediator;
    }
}

class ConcreteMediator implements Mediator {
    public void send(String message, Colleague sender) {
        // coordinate other colleagues
    }
}

class ColleagueA extends Colleague {
    ColleagueA(Mediator mediator) {
        super(mediator);
    }

    void send(String message) {
        mediator.send(message, this);
    }
}
```

Where to apply: Chat rooms • Complex GUI dialogs • Air-traffic/control systems • Workflow coordination • Reducing many-to-many dependencies

Typical scenarios

- Chat: User A → ChatRoom → Users B/C
- GUI: components → DialogMediator
- Airport: aircraft → control tower → aircraft

3. Observer Pattern

What it solves: Defines a one-to-many dependency: when a subject changes, its registered observers are notified.

Structure / Java template

```
interface Observer {
    void update();
}

interface Subject {
    void addObserver(Observer observer);
    void removeObserver(Observer observer);
    void notifyObservers();
}

class ConcreteSubject implements Subject {
    private List<Observer> observers = new ArrayList<>();
    private int state;

    public void addObserver(Observer o) {
        observers.add(o);
    }

    public void removeObserver(Observer o) {
        observers.remove(o);
    }

    public void notifyObservers() {
        for (Observer o : observers)
            o.update();
    }

    void setState(int state) {
        this.state = state;
        notifyObservers();
    }
}
```

```

    int getState() {
        return state;
    }
}

```

Where to apply: Event/listener systems • Stock/price dashboards • Weather apps • Publish-subscribe notifications • MVC model → views

Typical scenarios

- Channel: new video → subscribers
- Stock: price → chart + alerts
- GUI: event → registered listeners

4. State Pattern

What it solves: Moves state-specific behavior into separate state classes. The context delegates behavior to its current state.

Structure / Java template

```

interface State {
    void handle(Context context);
}

class StateA implements State {
    public void handle(Context context) {
        System.out.println("State A behavior");
        context.setState(new StateB());
    }
}

class StateB implements State {
    public void handle(Context context) {
        System.out.println("State B behavior");
    }
}

class Context {
    private State state;

    Context(State state) {
        this.state = state;
    }

    void setState(State state) {
        this.state = state;
    }

    void request() {
        state.handle(this);
    }
}

```

Where to apply: Vending machines • Order/payment lifecycles • ATM workflows • Media players • Network protocol states

Typical scenarios

- Order: Pending → Shipped → Delivered
- Player: Playing / Paused / Stopped
- ATM: NoCard → CardInserted → Authenticated

5. Strategy Pattern

What it solves: Defines a family of interchangeable algorithms and lets a context use one without knowing its implementation details.

Structure / Java template

```

interface Strategy {
    void execute();
}

```

```

}

class StrategyA implements Strategy {
    public void execute() {
        System.out.println("Algorithm A");
    }
}

class StrategyB implements Strategy {
    public void execute() {
        System.out.println("Algorithm B");
    }
}

class Context {
    private Strategy strategy;

    void setStrategy(Strategy strategy) {
        this.strategy = strategy;
    }

    void executeStrategy() {
        strategy.execute();
    }
}

```

Where to apply: Payment methods • Sorting algorithms • Routing/navigation • Compression/encryption choices • Pricing/recommendation algorithms

Typical scenarios

- Payment: Card / bKash / PayPal
- Sorting: QuickSort / MergeSort / HeapSort
- Navigation: Fastest / Shortest / Scenic

6. Template Method Pattern

What it solves: Defines the skeleton of an algorithm in an abstract class. Subclasses customize selected steps while the overall sequence stays fixed.

Structure / Java template

```

abstract class Beverage {

    // Template Method
    final void prepare() {
        boilWater();
        brew();
        pourIntoCup();
        addExtras();
    }

    void boilWater() {
        System.out.println("Boiling water");
    }

    abstract void brew();

    void pourIntoCup() {
        System.out.println("Pouring into cup");
    }

    abstract void addExtras();
}

class Tea extends Beverage {
    void brew() {
        System.out.println("Steeping tea");
    }

    void addExtras() {
        System.out.println("Adding lemon");
    }
}

class Coffee extends Beverage {
    void brew() {

```

```
        System.out.println("Brewing coffee");
    }

    void addExtras() {
        System.out.println("Adding milk");
    }
}
```

Where to apply: Data-processing pipelines • Report generation • File parsing • Build/deployment workflows • ML pipelines

Typical scenarios

- Beverage: boil → brew → pour → extras
- Parser: open → parse → validate → close
- Report: header → body → footer

Exam Identification Cheat Sheet

- Command:** Requests, buttons, undo/redo, queues, actions that should become objects.
- Mediator:** Many objects communicate directly and coupling is becoming complicated.
- Observer:** One object changes and multiple dependents must automatically be notified.
- State:** The same object behaves differently depending on its current state.
- Strategy:** Several interchangeable ways/algorithms can perform the same task.
- Template Method:** Subclasses follow the same algorithm structure but customize particular steps.

Key Comparisons

Comparison	Difference
Command vs Strategy	Command = WHAT action/request. Strategy = HOW to perform a task.
Observer vs Mediator	Observer = notification. Mediator = coordination.
State vs Strategy	State = behavior changes because the object's condition changes. Strategy = selected interchangeable a
Strategy vs Template Method	Strategy uses composition and replaces an algorithm. Template Method uses inheritance and fixes the al

Memory shortcut: Command = WHAT action? • Mediator = WHO coordinates? • Observer = WHO needs to know? • State = WHAT condition? • Strategy = HOW? • Template Method = WHICH steps are fixed/variable?